Studio Nx

JS 166 — selectTab()

2026-10-15

JS 166 — `selectTab()` (복구 화면의 탭 고르기)

항목	내용
파일	`studionx/frontend/static/js/core/recover.js`
작업	P1-28 계정 찾기 화면: JS 두 번째 함수
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	165 `showView()`
다음	탭 클릭과 화살표 키 이동(이 함수를 부르는 처리기), 그다음 열기 연결

1. 하는 일

복구 화면의 두 탭(`password`, `email`) 중 하나를 고릅니다.

하는 일	설명
탭 표시	고른 탭만 `aria-selected="true"`와 `tabindex="0"`, 나머지는 `"false"`와 `"-1"`
패널 전환	짝이 되는 패널(`recover-panel-<이름>`)만 보이고 나머지는 숨김
값 보존	입력칸의 값과 오류 문구는 **건드리지 않음**: 탭을 오가도 적던 이메일이 남습니다
커서	`focus`가 `true`이거나, 숨겨질 패널의 칸에 커서가 있었으면 고른 탭으로 옮김
반환	바꿨으면 `true`, 모르는 이름, 모달 없음, 탭이나 패널이 없으면 아무것도 바꾸지 않고 `false`

2. 코드

```
export const RECOVER_TABS = ['password', 'email'];

export function selectTab(modal, name, focus) {
  if (!modal || RECOVER_TABS.indexOf(name) < 0) return false;
  const tabs = modal.querySelectorAll('[role="tab"]');
  const panels = modal.querySelectorAll('[role="tabpanel"]');
  const tabId = 'recover-tab-' + name;
  const panelId = 'recover-panel-' + name;

  let chosen = null;
  let chosenPanel = null;
  for (const tab of tabs) if (tab.id === tabId) chosen = tab;
  for (const panel of panels) if (panel.id === panelId) chosenPanel = panel;
  if (chosen === null || chosenPanel === null) return false;

  for (const tab of tabs) {
    const on = tab === chosen;
    tab.setAttribute('aria-selected', on ? 'true' : 'false');
    tab.setAttribute('tabindex', on ? '0' : '-1');
  }
  const active = document.activeElement;
  let lostFocus = false;
  for (const panel of panels) {
    if (panel !== chosenPanel && active && panel.contains(active)) lostFocus = true;
```

```
    panel.hidden = panel !== chosenPanel;
  }
  if (focus === true || lostFocus) chosen.focus();
  return true;
}
```

최신 문법으로 쓰면 (공부용)

```
export const selectTab = (modal, name, focus = false) => {
  if (!modal || !RECOVER_TABS.includes(name)) return false;
  const tabs = [...modal.querySelectorAll('[role="tab"]')];
  const panels = [...modal.querySelectorAll('[role="tabpanel"]')];
  const chosen = tabs.find((t) => t.id === `recover-tab-${name}`);
  const chosenPanel = panels.find((p) => p.id === `recover-panel-${name}`);
  if (!chosen || !chosenPanel) return false;

  tabs.forEach((t) => {
    t.setAttribute('aria-selected', String(t === chosen));
    t.tabIndex = t === chosen ? 0 : -1;
  });
  const lostFocus = panels.some((p) => p !== chosenPanel && p.contains(document.activeElement));
  panels.forEach((p) => { p.hidden = p !== chosenPanel; });
  if (focus === true || lostFocus) chosen.focus();
  return true;
};
```

최신 문법	실제 코드	뜻
`tabs.find(…)`	`for` 반복에서 변수에 담기	이름이 같은 탭을 찾습니다
`panels.some(…)`	`for` 안의 `if`로 `lostFocus` 표시	숨겨질 패널 안에 커서가 있는지 봅니다
`String(t === chosen)`	`on ? 'true' : 'false'`	참, 거짓을 글자로 바꿉니다
`t.tabIndex = …`	`setAttribute('tabindex', …)`	같은 일입니다

문법 풀이

식	뜻
`aria-selected`	화면 낭독기에 "이 탭이 선택됨"을 알립니다
`tabindex="0"` / `"-1"`	**Tab 키가 선택된 탭 하나에만 멈추게** 합니다. 나머지 탭은 화살표 키로 옮깁니다(WAI-ARIA 탭 패턴). 탭이 둘뿐이어도 이 패턴을 지켜서, 다음 단계의 화살표 키 이동과 맞물립니다
`panel.contains(active)`	지금 커서가 그 패널 안의 요소인지 확인합니다

3. 왜 이렇게

선택	이유
값과 오류 문구를 지우지 않음	사용자가 Password 탭에 이메일을 적다가 Email 탭을 보고 돌아와도 적은 것이 남습니다. 비우는 일은 모달을 닫을 때의 몫입니다
숨겨질 패널의 칸에 커서가 있었으면 탭으로 옮김	시험을 하다 발견했습니다(아래 §5). 숨은 칸에서는 브라우저가 커서를 빼앗아 `body`로 보내서, 키보드 사용자가 위치를 잃습니다
`focus === true`로 엄격하게 비교	`1`이나 `'false'` 같은 값이 실수로 커서를 옮기지 않게 합니다

선택	이유
`focus` 인자를 따로 둠	클릭은 브라우저가 이미 탭에 커서를 주고, 키보드 이동은 다음 탭으로 커서를 옮겨야 해서 부르는 쪽이 정합니다
탭과 패널이 모두 있어야 진행	둘 중 하나가 없는데 진행하면 탭 표시만 바뀌고 패널은 그대로인 어긋난 상태가 됩니다
모르는 이름은 `false`	오타로 모든 탭이 선택 해제되는 일을 막습니다

4. 시험 (실제 서버, 헤드리스 Chrome, 40개 통과)

`home.html`과 `join.html`에서 같은 시험을 했습니다.

확인	결과
시작 상태: Password 탭 선택, Email 패널 숨음	✓ ✓
`email` 선택: `aria-selected`, `tabindex`, 패널이 **실제로**(`getComputedStyle`) 바뀜	✓ ✓
이미 그 탭이면 같은 결과	✓ ✓
탭을 오가도 두 칸에 적은 값이 남음	✓ ✓
오류 문구도 건드리지 않음	✓ ✓
`focus` 인자가 없거나 `false`, `1`이면 커서를 옮기지 않음	✓ (6개)
`focus=true`면 고른 탭으로 커서	✓ ✓
숨겨질 패널의 칸에 있던 커서는 고른 탭으로 옮겨져 사라지지 않음	✓ ✓
같은 탭을 다시 고르면 칸의 커서를 그대로 둠	✓ ✓
모르는 이름 7종은 `false`이고 안 바뀜	✓ (14개)
모달이 `null`이면 `false`	✓ ✓
탭이나 패널이 없는 모달은 `false`이고 아무것도 안 바뀜	✓
내보내는 상수	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
`aria-selected`를 반대로	2개 실패
`tabindex` 갱신 제거	2개 실패
패널 전환 제거	2개 실패
입력칸 값 비우기	2개 실패
`focus`의 엄격한 비교를 느슨하게	2개 실패
`focus` 인자 무시	2개 실패
잃는 커서 처리 제거	**6개 실패**
탭만 있고 패널이 없는 경우를 진행	1개 실패
이름 검사 제거	**실패 없음**

마지막은 시험의 빈틈이 아니라 중복 방어입니다. 모르는 이름이면 `recover-tab-<이름>`인 요소가 없어서 바로 아래의 `chosen === null` 검사가 같은 경우를 `false`로 막기 때문입니다. 이름 검사는 요소를 찾으러 가기 전의 빠른 반환입니다.

5. 시험이 잡은 문제: 숨은 칸에서 커서가 사라짐

"이미 그 탭이면 같은 결과" 시험이 처음에 실패했습니다. 두 번 부른 상태를 비교했는데 한 가지만 달랐습니다. **커서(`document.activeElement`)였습니다.** 첫 호출 직후에는 `recover-password-email`이었다가, 잠시 뒤에는 아무 요소도 아닌 상태(`body`)가 되어 있었습니다.

- **원인:** Email 탭으로 바꾸면서 Password 패널을 숨겼는데, 커서가 그 패널의 이메일 칸에 있었습니다. 브라우저는 숨겨진 요소의 커서를 빼앗고(`body`로 보냄), 그것이 화면에 그려진 뒤에야 반영됩니다. 그래서 첫 시험 값은 옛 상태였습니다.
- **영향:** 키보드로 쓰는 사람이 Password 칸에 있다가 탭이 바뀌면 커서가 사라져 Tab 키를 눌러도 문서 맨 위부터 다시 시작합니다.
- **고침:** 패널을 숨기기 전에 "커서가 숨겨질 패널 안에 있는지" 확인하고, 그렇다면 고른 탭으로 커서를 옮깁니다. 시험도 같은 경우를 확인하게 더했고(위 표), 이 경우를 지우면 6개가 실패합니다.

6. 아직 이 함수를 부르는 곳이 없습니다

탭을 눌러도 아직 바뀌지 않습니다. 다음 함수가 클릭과 키보드(좌우 화살표, Home, End)를 이 함수에 연결합니다.